Airbus Ship Detection: Traditional ML vs. Convolutional Neural Networks

UE24CS352A – Machine Learning | Mini-Project
Team 9 | Project No. 35 | Rohan Suresh (PES1UG24CS383) · Satvik L (PES1UG24CS423)

1. Problem statement

Maritime monitoring (illegal fishing, piracy, accident response) needs ships to be found in satellite imagery automatically. Following the Stanford CS229 project *Airbus Ship Detection – Traditional v.s. Convolutional Neural Network Approach* (Chen, Zheng, Zhou, 2018), we treat it as **binary image classification**: given a satellite image, decide whether it contains at least one ship. The aim is to compare traditional machine-learning classifiers on hand-engineered features against convolutional neural networks that learn their features from pixels. It is hard because ships are tiny relative to the image and scenes vary widely (open sea, docks, clouds, haze, wakes).

2. Dataset

Source: the Kaggle **Airbus Ship Detection Challenge** – 192,555 labelled 768×768 RGB satellite images (31 GB) and a CSV of run-length-encoded ship masks; an image with no mask has no ship (22.1% of all images contain one). The reference report used a 15k-image subset; with a 1 GB download budget we used **3,829 images**: a ready-made 494 MB subset of the competition's training images plus 515 images sampled uniformly at random (Kaggle rate-limits per-image downloads). Every label comes from the official competition CSV. Our subset is ship-enriched – **67.2% ship / 32.8% no ship** – so the trivial 'always predict ship' baseline is 67.2%, and we report precision, recall and specificity next to accuracy. Images are resized to 256×256×3 and split with stratification in the report's 7:3:5 ratio: **1,786 train / 766 dev / 1,277 test**. The test set is used only for the final numbers.

3. Approach

Feature engineering (traditional models). Three global descriptors per image: **His** – an 8×8×8 colour histogram in HSV space (512 values, colour); **Ha** – 13 Haralick statistics of the grey-level co-occurrence matrix (texture); **Hu** – the 7 Hu moments (shape). As the 'no feature engineering' baseline the image is shrunk to 32×32×3 and flattened (3,072 raw pixel values). Features are standardised using training-set statistics.

Traditional classifiers. Linear Discriminant Analysis, k-Nearest Neighbours (k = 5, Euclidean), Gaussian Naive Bayes, Random Forest (70 trees) and an RBF-kernel SVM, each trained on all 8 feature combinations (40 experiments), plus stratified 10-fold cross-validation on train + dev to measure stability.

SIM-CNN (from scratch). 4 × [3×3 convolution → ReLU → 2×2 max-pool] with 16/32/64/128 filters, then FC(128) → dropout → FC(1) → sigmoid; 4,292,001 trainable parameters, identical to the report. Trained with augmentation (flips, rotation, shift, zoom, shear, brightness). **TL-CNN (transfer learning).** ImageNet-pretrained DenseNet-169 with all weights frozen → global max-pool → BatchNorm → dropout → FC(128) → FC(1) → sigmoid; only the 216,577-parameter head is trained (also identical to the report).

Training and evaluation. Binary cross-entropy loss, Adam (lr 10^{-3}, halved when the dev loss stalls), batch size 64, 30 epochs, keeping the checkpoint with the lowest dev loss. Because the classes are imbalanced, the decision threshold is chosen by scanning 0.1–0.9 on the dev set instead of assuming 0.5.

4. Implementation overview

Python 3.13; scikit-learn for the traditional models, OpenCV and mahotas for features, PyTorch/torchvision for the CNNs; trained on a laptop CPU. The code is a small package (`src/`) with one module per stage: `download_data` (subset download, resize, labels, split) → `features` → `train_traditional` → `train_cnn` → `evaluate_cnn` → `make_figures`, plus `demo` for live prediction. `run_all.py` runs every stage; all settings live in `config.py`; random seeds are fixed and the exact image list and split are committed (`data/labels.csv`). Since the DenseNet backbone is frozen, its output for each image is computed once and cached, so the TL-CNN head trains in seconds.

5. Results

Table 1. Test accuracy of the traditional models for each feature set (1,277 test images; best value in bold).

	His	Ha	Hu	His+Ha	Ha+Hu	Hu+His	All three	Raw pixels
LDA	0.748	0.784	0.683	0.849	0.785	0.756	0.854	0.537
KNN	0.785	0.829	0.674	0.838	0.825	0.795	0.839	0.735
NB	0.526	0.734	0.672	0.525	0.676	0.531	0.532	0.625
RF	0.915	0.854	0.733	0.918	0.854	0.913	**0.919**	0.822
SVM	0.761	0.832	0.680	0.848	0.825	0.762	0.850	0.808

Table 2. All models on the same test set (traditional models use all three descriptors).

Model	Accuracy	Precision	Recall	Specificity / notes
LDA (all features)	85.4%	87.0%	92.0%	10-fold CV 83.4% ± 2.5%
KNN (all features)	83.9%	85.6%	91.4%	10-fold CV 82.9% ± 2.4%
NB (all features)	53.2%	91.4%	33.4%	10-fold CV 51.5% ± 2.8%
RF (all features)	91.9%	93.2%	94.9%	10-fold CV 92.4% ± 1.7%
SVM (all features)	85.0%	84.3%	95.5%	10-fold CV 84.6% ± 2.5%
TL-CNN (thr 0.5)	95.3%	96.3%	96.7%	specificity 92.4%, AUC 0.982
SIM-CNN (thr 0.6)	88.0%	91.1%	91.0%	specificity 81.9%, AUC 0.928
Always predict 'Ship'	67.2%	67.2%	100.0%	specificity 0% (baseline)

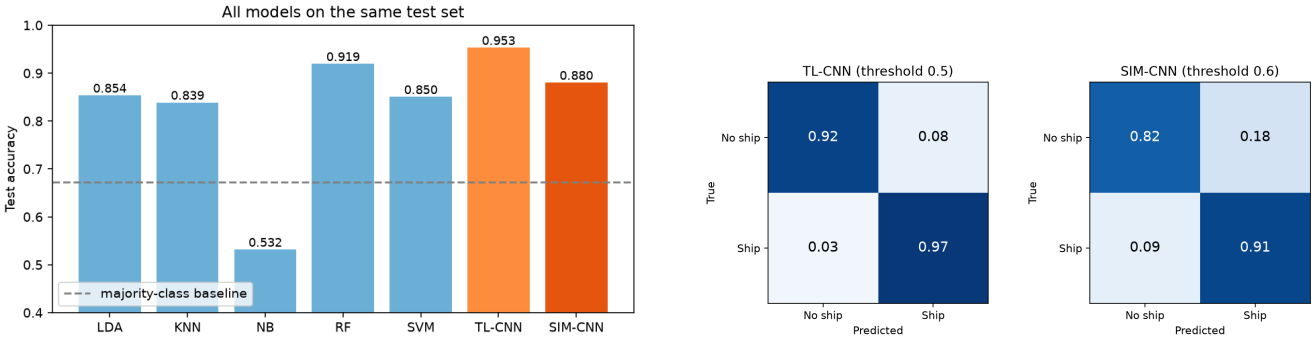


Figure 1. Left: test accuracy of all models against the majority-class baseline. Right: normalised confusion matrices of the two CNNs.

6. Conclusions

- **Random Forest is the best traditional model** (91.9% test accuracy, 10-fold CV 92.4% ± 1.7%), in agreement with the reference report (93%).
- **Feature engineering matters:** going from raw pixels to the three descriptors changed accuracy by LDA +31.6, KNN +10.4, NB -9.3, RF +9.8, SVM +4.2 points. It helped LDA, KNN, RF, SVM but hurt NB. More features are not always better – the 512 correlated histogram bins break Naive Bayes's independence assumption, as the report also observed.
- **CNNs:** TL-CNN (95.3%) scored higher than SIM-CNN (88.0%) on our test set; the best model overall was TL-CNN (95.3%). Unlike the report, transfer learning beat the from-scratch network. A likely reason is data size: we trained on 1,786 images instead of 7,000, and a 4.3 M-parameter network trained from scratch needs far more data than a small head on features pre-learned from 1.2 M ImageNet images. SIM-CNN also scored below Random Forest.
- **Limitations:** a small, ship-enriched subset (so absolute numbers are not directly comparable with the report's 84%-no-ship data), CPU-only training, a single run per model, SIM-CNN's dev loss was still falling at epoch 30 (it had not converged), and image-level classification only. **Future work:** train on the full dataset, fine-tune the backbone, and segment/localise the ships.